Name: Disha Andre

PRN: 123B1B078

Assignment No. 3

Title:

You find yourself in a labyrinthine dungeon with multiple rooms and connecting passageways. Each passage has a specific time to travel, and some passages are blocked due to cave-ins. Starting from the main entrance, you must find the shortest escape routes to every other room in the dungeon. Use Dijkstra's algorithm to determine the quickest time required to reach each room from the entrance. If any room is isolated and unreachable, mark it as -1. Test this with a dungeon of at least 15 rooms and random travel times for each accessible passage.

Objectives:

1. To study shortest path problems in weighted graphs.
2. To understand and implement Dijkstra's Algorithm.
3. To compute minimum travel time from a source node to all other nodes.
4. To identify unreachable nodes in a graph.

Course Outcome Mapping:

CO2: Analyze the efficiency of algorithms using Greedy strategy.

Theory:

In real-world scenarios like navigation systems, network routing, and game development, finding the shortest path between locations is very important. This problem can be represented using a graph, where:

- Rooms are represented as nodes (vertices)
- Passageways are represented as edges
- Travel time is represented as edge weight

Some paths may be blocked, meaning no edge exists between certain nodes.

To solve this problem efficiently, Dijkstra's Algorithm is used. It is a Greedy Algorithm that finds the shortest path from a source node to all other nodes in a graph with non-negative edge weights.

Dijkstra's Algorithm:

The algorithm works by selecting the node with the smallest known distance and updating its neighboring nodes.

Steps:

1. Assign distance value = ∞ (infinity) to all nodes except the source node (distance = 0).
2. Mark all nodes as unvisited.
3. Select the unvisited node with the smallest distance.
4. Update the distance of its neighboring nodes:
   New Distance = Current Distance + Edge Weight
5. If the new distance is smaller, update it.
6. Mark the current node as visited.
7. Repeat until all nodes are visited or no reachable nodes remain.

Example:

Consider a dungeon with 5 rooms:

From	To	Time
0	1	4
0	2	1
2	1	2
1	3	1
2	3	5
3	4	3

Step-wise Execution:

- Start from Room 0
- Distance[0] = 0

Room	Shortest Time
0	**0**
1	**3**
2	**1**
3	**4**
4	**7**

Time Complexity:

- Using Min Heap: O((V + E) log V)
- Where:
    - V = Number of vertices (rooms)
    - E = Number of edges (passages)

This makes Dijkstra's Algorithm efficient for large graphs.

Code:

```
import random
import heapq
# Generate adjacency list directly
def generate_graph(n, probability=0.3):
    graph = {i: [] for i in range(n)}
    for i in range(n):
        for j in range(i + 1, n):
            if random.random() < probability:
                weight = random.randint(1, 20)
                graph[i].append((j, weight))
                graph[j].append((i, weight))
    return graph
# Dijkstra's Algorithm using Min Heap
def dijkstra(graph, start):
    n = len(graph)
    distances = [float('inf')] * n
    distances[start] = 0
    min_heap = [(0, start)]
    while min_heap:
        current_dist, u = heapq.heappop(min_heap)
        if current_dist > distances[u]:
```

```
            continue
        for v, weight in graph[u]:
            new_dist = current_dist + weight
            if new_dist < distances[v]:
                distances[v] = new_dist
                heapq.heappush(min_heap, (new_dist, v))
    # Mark unreachable rooms as -1
    return [-1 if d == float('inf') else d for d in distances]
# Main Execution
if __name__ == "__main__":
    NUM_ROOMS = 15
    graph = generate_graph(NUM_ROOMS)
    print("Adjacency List (Room: [(Connected Room, Time), ...]):\n")
    for room in graph:
        print(f"{room}: {graph[room]}")
    result = dijkstra(graph, 0)
    print("\nShortest time from Entrance (Room 0):\n")
    for i, time in enumerate(result):
        print(f"Room {i}: {time}")
```

Output:

```
PS C:\Users\SUJAL\Desktop\Disha\UI-UX\mindbloom> python -u "c:\Users\SUJAL\Desktop\Disha\DAA\dijkstra.py"
Adjacency List (Room: [(Connected Room, Time), ...]):

0: [(10, 7), (12, 20)]
1: [(4, 7), (8, 20), (10, 8), (11, 19), (12, 2), (14, 14)]
2: [(5, 10), (6, 4), (12, 7)]
3: [(4, 20), (7, 2), (8, 1), (12, 6), (14, 13)]
4: [(1, 7), (3, 20), (5, 14), (14, 15)]
5: [(2, 10), (4, 14), (6, 1), (7, 11)]
6: [(2, 4), (5, 1), (11, 3)]
7: [(3, 2), (5, 11), (14, 1)]
8: [(1, 20), (3, 1), (11, 18), (12, 14)]
9: []
10: [(0, 7), (1, 8), (11, 9), (13, 6)]
11: [(1, 19), (6, 3), (8, 18), (10, 9)]
12: [(0, 20), (1, 2), (2, 7), (3, 6), (8, 14)]
13: [(10, 6)]
14: [(1, 14), (3, 13), (4, 15), (7, 1)]
```

```
Shortest time from Entrance (Room 0):

Room 0: 0
Room 1: 15
Room 2: 23
Room 3: 23
Room 4: 22
Room 5: 20
Room 6: 19
Room 7: 25
Room 8: 24
Room 9: -1
Room 10: 7
Room 11: 16
Room 12: 17
Room 13: 13
Room 14: 26
```

Conclusion:

Dijkstra's Algorithm efficiently computes the shortest path from a source node to all other nodes in a graph with non-negative edge weights. In this assignment, it successfully determines the minimum time required to reach each room in the dungeon.

The algorithm also identifies unreachable rooms by marking them as -1. With a time complexity of O((V + E) log V), it is suitable for solving large-scale shortest path problems in real-world applications such as navigation systems, networking, and game environments.